

k-NN: k Nearest Neighbor

Classification, Regression

Classification: eval k-NN and classify data point according to majority occurrence

Regression: average k closest samples

good for: small data sets

- supervised and non-parametric
- can learn highly non-linear class boundaries
- ~~efficient~~ requires data orgn/storage for efficiency
- Lazy Learner
- distance metric and dimensions chosen carefully

k = how many nearest neighbors are evaluated for classification

large k → large bias, inaccurate estimation
small k → large variance, unreliable classification

k-means

Clustering

1. k values as init centroids
2. assign data point according to distance to centroid
3. re-compute centroid
4. repeat until change in centroid $< \epsilon$

+

- fairly efficient $O(nkt)$
- simple to implement

=

- need numeric data
- prone to noise and outliers
- needs convex clusters
- often converges to local optimum based on initial centroids

samples ↓ clusters
iterations ↓

Elbow Method (to choose k)

compute within cluster sum of squares WCSS, i.e. cluster coherence
→ dist between points within cluster to each other

- too large k has diminishing returns in improving WCSS
- recommendation to use k at elbow

k = how many centroids to use
 ϵ = min change in centroid position



~~Scatter~~ Silhouette Coefficient (to choose k)

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

a(i) avg dist of point i to all other points in cluster

b(i) avg dist of point i to all other points in other nearest cluster

$S(i) \approx 1$ → well assigned

$S(i) \approx 0$ → near boundary

$S(i) < 0$ → possibly misassigned

- compute ~~scat~~ average s for several k

- use k with highest average

k-medoids (k-means variant) / PAM Algorithm (Partitioning around Medoid)

Clustering

medoid: feature vector with minimal cumulative dissimilarity to all other features within cluster

works same as k-means but with fractions to compute cumulative dist to medoid (quality / cost) and improvement of new possible medoid in comparison

=

- high complexity: $O(k(n-k)^2)$

+

- guaranteed global optimum by trying all permutations
- for non-euclidean data
- less sensitive to noise and outliers

Gaussian Mixture Model (GMM)

Clustering

- like k-Means but uses Gaussian normal distribution as "centroid"

uses Expectation Maximization (EM) Algo

1. randomly choose mean μ_c , variance σ_c^2 and weight π_c for each cluster c
2. E-Step: Expectation (?)
↳ update probability estimation (responsibility) r_{ic} that x_i belongs to cluster c
3. M-Step: Maximization
↳ update μ_c , σ_c^2 and π_c for each cluster based on r_{ic}
4. repeat until model converges based on (log) likelihood L and min difference in L improvement (→ L always increases)

→ visually "moves" Gaussian distribution to center ~~there~~ above cluster

Bayesian Information Criterion (BIC) to choose k

$$BIC = -2 \log L + p \log n$$

→ lower BIC = better
tradeoff between fit and complexity

L : max likelihood of model
 p : total num of free model params
 n : num of data points

- compute BIC for multiple GMM k s
- choose model with lowest BIC

⊕ flexible
explain observations
using latent "hidden"
groupings of data

Multidimensional / Multivariate GMM

- data points are d -dim vectors
- $\vec{\mu}_c$ is d -dim vector
- variance is $d \times d$ -dim Covariance matrix
- uses multivariate Gaussian distribution
- same algo as before but with vectors where applicable

Common Covariance structures:

- Spherical: circular, ^{same} spread in all directions
param per cluster: 1
- Diagonal: axis-aligned ellipse/ellipsoid
param per cluster: d
- Full: arbitrary ellipse/ellipsoid
ppc: $d(d+1)/2$
- Tied: all same shape
param for whole model: $d(d+1)/2$

Decision Tree

Classification, Regression

for Classification:

Gini Impurity (per leaf) $1 - \sum_{i=1}^c p_i^2$

e.g. $\boxed{1} \begin{bmatrix} \frac{4}{3} & \frac{2}{3} \\ 3 & 1 \end{bmatrix} \quad \boxed{2} \begin{bmatrix} \frac{4}{2} & \frac{2}{2} \\ 2 & 1 \end{bmatrix}$
 $\boxed{1} G_1 = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 0.375$
 $\boxed{2} G_2 = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 0.444$

Total Gini Impurity for Split

$$\frac{\text{num elems split}}{\text{total elems}} \cdot G_1$$

sum of all leaves weighted by num elements

$$\text{Total } G_1 = \frac{4}{7} \cdot 0.375 + \frac{3}{7} \cdot 0.444 = 0.405$$

- for continuous features, this is done for each threshold and threshold with best Gini Impurity is chosen
- lowest Gini Impurity is best

subsequent leaves only use the data that lands on the given branch

Tree Construction: for each feature

calc Gini Impurity for each split
choose feature split with best overall Gini Impurity
repeat until max-depth reached

Simple Linear Regression

Regression : statistical of estimating relationship between dependent (y) and one or more independent variables (x)

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

$\uparrow$ $\uparrow$ $\uparrow$
y-intercept slope stochastic error term

- forecast/predict dependent given independent variable
- quantify relationship strength

Ordinary Least Squares (OLS)
to estimate coefficients

$\rightarrow b_0$ as estimation for β_0
 b_1 as estimation for β_1

Multiple Linear Regression

$$y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \quad \epsilon = \begin{pmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{pmatrix} \quad \beta = \begin{pmatrix} \beta_0 \\ \vdots \\ \beta_k \end{pmatrix}$$

equiv. y-intercept

$$X = \begin{pmatrix} 1 & x_{11} & \dots & x_{1k} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \dots & x_{nk} \end{pmatrix}$$

$$y = X\beta + \epsilon$$

this is what we want to find
 $\rightarrow$ coefficient for each feature

Polynomial Regression

- model is not linear and instead polynomial to better fit data

Decision Trees contd.

Information Gain via Entropy $\rightarrow H(x) = - \sum p_i \log_2(p_i)$

- higher info gain = more useful split

$$IG = H(\text{Parent}) - \sum \left(\frac{N_{\text{child}}}{N_{\text{parent}}} H(\text{Child}) \right)$$

Regression Tree

for each feature
for each split (or threshold)
compute sum of squared residuals based on all data points in that split in relation to target
choose split with lowest SSR within feature
choose feature with lowest SSR compared to other features

Ensemble Learning $\rightarrow \oplus$

Bagging (Bootstrap Aggregation)

- combines multiple high variance decision trees

- better accuracy (wisdom of the crowd)
- less overfitting: Bagging reduces variance, Boosting reduces bias
- more robust for diff types of errors
- versatile: for both classification and regression

- train various trees on Bootstrap sample
- combine results via Majority vote (class) / avg function (regress)

but: trees correlated
strong features will dominate
 $\rightarrow$ improves generalization, reduces overfitting

Random Forest

- same as Bagging but at each node only uses random subset of features for splitting

Extra Trees

- similar to Random Forest but no bootstrapping and random splits instead of best splits

Extra Trees	Feature	Random Forest	Extra Trees
→ impact on bias	Bootstrapping	Yes	No
	Split Selection	Best	Random
	Variance Reduction	High	Even Higher
	Bias	Slightly Lower	Slightly Higher
	Speed	Slower	Faster

- Random Forest > Extra Trees for accuracy
- Extra Trees > Random Forest for noisy data (overfitting issue)

Optimal Classifier

Classification

$$u_{\lambda}(c) = \sum_{k=1}^K r_{\lambda k} p_k p(c|\Omega_k)$$

but: prob. distribution hard and expensive to get, needs large complete data set
rejection class hard to define

↑ probability density functions
↑ r is cost

$0 \leq r_{\lambda k} < r_{0k} < r_{\lambda k} \quad \lambda \neq k$
↑ predicted class
↑ actual class

(Optimal) Bayes Classifier

- no rejection allowed

→ minimize mean cost of classification

$$p(\Omega_{\lambda}|c) = \frac{P_{\lambda} p(c|\Omega_{\lambda})}{p(c)}$$

- Bayes requires complete statistical information (i.e. no missing data)

Naive Bayes Classifier

- assumes statistically independent features, so no division

$$p(c|\Omega_{\lambda}) = \prod_{i=1}^d p(c_i|\Omega_{\lambda})$$

Num needed for $p(c|\Omega_{\lambda})$
→ Optimal: $(\prod \text{values of each feature}) \times 2$

Laplace Correction to avoid 0 Product, +1

→ Naive: $\sum (\text{num feature values} \times 2)$
num feature values meaning how many different values

Gaussian Naive Bayes Classifier

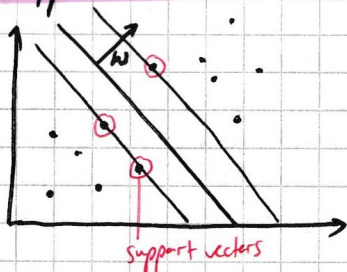
$$p(c_i|\Omega_{\lambda}) = \frac{1}{\sqrt{2\pi}\sigma_{i,\lambda}} e^{-\frac{(c_i - \mu_{i,\lambda})^2}{2\sigma_{i,\lambda}^2}}$$

← one dim Gaussian distribution

→ Naive: Spam Filtering

⊕ fast, trained per user
good at avoiding false positives

Support Vector Machines



- Feature Transformation: map data to higher dim using non-linear feature transform to make data linearly separable

- Kernel Trick: using kernel function, vectors do not have to be actually transformed (expensive, sometimes impossible)

- minimization problem

- classify with (rejections) or without (no rejections) threshold

Boosting

- train sequence ~~of~~ of models that sequentially improve
- many simple high bias models into one complex classifier
 - avoid underfitting by reducing bias
 - avoid overfitting by controlling variance

Ada Boost (Adaptive Boosting)

- iteratively add classifiers, each time re-weighting data set to target errors
 - weak learners: decision stumps (depth=1)
- Classification,
Regression

Gradient Boosting

init: avg target
compute residuals based on avg
create regression tree based on residuals
re-compute ~~the~~ residuals by "predicting" each data point and computing new residual
create new regression tree based on new residuals

$\boxed{\text{init}} + \text{learning_rate} \cdot \boxed{\text{regression}} + \dots$
hyperparam

stopping criterion: param runs trees
residuals reduction non-significant

→ for Regression

for Classification: uses $\log(\text{odds})$ and sigmoid function
convert yes and no to 1 and 0
residuals based on $\log(\text{odds}) \rightarrow \text{sig}(x) \rightarrow \text{residuals}$
outputs probability

XG Boost

init: constant 0.5
same as Gradient Boosting but build tree by splitting features and computing similarity of leaves in comparison to $\text{Similarity}(\text{root})$

$$\text{Similarity} = \frac{(\sum \text{residuals})^2}{\text{residuals} + \lambda}$$

- also uses pruning of tree leaves where $\text{Gain} < \gamma$ threshold

- λ regularization to reduce sensitivity to individual data point

Classification is the same but uses different similarity function

- Weighted Quantiles: focus on hardest to learn data set parts

Accuracy: num examples correctly classified $\frac{TP + TN}{TP + TN + FP + FN}$

Precision: fraction of all labelled positives that were correctly labelled as such $\frac{TP}{TP + FP}$

Recall: True Positive Rate, percentage of all positives that were found $\frac{TP}{TP + FN}$

Sensitivity: = Recall, correctly detect affected patient who actually has condition low type II error

Specificity: correctly reject healthy patients, True Negative Rate low type I error rate $\frac{TN}{TN + FP}$

SPPIN: specific test when pos rules in

SNNOUT: sensitive test when neg rules out